



EXAM GRADE PREDICTOR

Project Documentation, Technical Guide & Learning Report

A frontend-only academic planning application for weighted grade calculation, target planning, What-If simulation, and scenario comparison.

Created by	Ann Noor Muhammad Mrinmoy
Project Type	Portfolio / University / Student Utility

Core stack: React 19 | Vite 8 | JavaScript | Modern CSS | Lucide React | localStorage

Executive Overview & Motivation

Start here if you want the project idea, purpose, and value in a few minutes.

Project Overview

Exam Grade Predictor is a polished, frontend-only academic planning application. It lets a student model a course using assessments such as Quiz, Assignment, Midterm, Project, Lab, Presentation, and Final Exam; calculate weighted academic performance; set a target result; and test future score combinations without confusing predictions with actual marks.

The important design decision is that the app does not behave like a basic marks calculator. It treats a course as a weighted system, separates completed work from pending work, and explains what score is mathematically required to reach a target.

Motivation

এই project-এর মূল motivation হলো student-দের একটা common problem solve করা: “এখন আমার performance আসলে কত?”, “Final-এ কত পেলে A- হবে?”, “Current average 82% হলেও course points কেন 41?”, “আর যদি Final-এ 70/80/90 পাই, outcome কী হবে?” - এগুলো manually calculate করতে গিয়ে weight, percentage আর future assumption mix হয়ে যায়।

এই application সেই confusion কমায়। Completed assessment থেকে earned course points বের করে, pending assessment আলাদা রাখে, target অনুযায়ী required score calculate করে, এবং What-If simulator দিয়ে future scenario test করতে দেয়।

Core idea in one sentence

A student enters the course structure once, then the app becomes a transparent mathematical planner for current performance and future targets.

What problem does it solve?

- Simple averaging error: Quiz 80%, Midterm 70%, Final 90% - এগুলোর weight different হলে direct average ভুল result দিতে পারে।
- Current Average vs Course Points confusion: completed work-এর average আর full course-এর earned points এক জিনিস না।
- Future prediction confusion: expected Final mark actual result নয় - তাই What-If data আলাদা রাখা দরকার।
- Target planning: A-, A, Pass বা custom percentage target করতে remaining work-এ কত average দরকার সেটা জানা।
- Scenario comparison: 70%, 80%, 90% Final assumption save করে outcome compare করা।

Who can use it?

User	Use Case
University student	Semester grade planning and Final target calculation
Project presenter	Portfolio demo, university project presentation, viva
Self-learner	React state, validation, localStorage, modular calculation logic শেখা
Instructor/demo audience	Weighted grading mathematics transparently দেখানো

Feature Map & User Experience

What the application can do from the user perspective.

Main Features

Feature	What it does	Why it matters
Course Setup	Course name, code, pass mark, grade scale	Saved/printed reports meaningful হয়
Dynamic Assessments	Add/edit/remove assessment, status Completed/Pending	Different course structures support করে
Weight Validation	Below 100, exactly 100, over 100, invalid states	Wrong course model silently normalize করে না
Current Performance	Current Average, Course Points, Completed Weight	Completed-work performance clear হয়
Target Planner	Target percentage or letter grade	Student goal mathematically define হয়
Required Score	Selected pending assessment-এ কত %/raw mark দরকার	Actionable target পাওয়া যায়
What-If Simulator	Pending marks simulate with slider/input/presets	Future scenarios compare করা যায়
Grade Possibility	Secured / Reachable / Perfect / Not Reachable	Which grades are still mathematically possible
Saved Scenarios	Save/load/duplicate/rename/delete/compare	Different plans preserve করা যায়
Export & Report	CSV, JSON backup, copy summary, print PDF	Portfolio and practical use
Theme & Responsive	Light/dark + mobile cards	Real app-like UX

Important UX rule

Predicted or simulated marks never become completed marks automatically. The user must explicitly change assessment status/data.

Technology Stack & Why Each Tool Was Used

The actual final package dependencies and their roles.

Technology Stack

Technology	Role in this project	Why chosen
React 19.2.8	Component-based UI, state, derived calculations	Interactive dashboard cleanly organize করা যায়
Vite 8.2.2	Development server and production bundler	Fast dev workflow, simple React setup
JavaScript (ES Modules)	Business logic + UI behavior	No TypeScript complexity; beginner-friendly
Modern CSS	Layout, theme, responsive cards, print styling	No heavy UI framework dependency
Lucide React 1.34.0	Consistent interface icons	Lightweight polished visual language
Browser localStorage	Scenarios, settings, theme, draft persistence	Backend/database ছাড়া persistence
Clipboard / Blob / Print APIs	Copy, CSV/JSON downloads, Save as PDF	Native browser features leveraged

Why React?

React ব্যবহার করা হয়েছে কারণ app-এ অনেক interconnected state আছে: assessments change হলে current average, target requirement, grade ladder, What-If projection - সব update হতে হয়। Component-based structure এ UI ভাগ করা যায়, আর useMemo দিয়ে derived mathematics centrally recalculate করা যায়।

Why Vite?

Vite development server দ্রুত, configuration কম, এবং React project-এর জন্য clean build pipeline দেয়। `npm run dev` development server চালায়, `npm run build` production bundle তৈরি করে, আর `npm run preview` build locally preview করে।

Why no backend or database?

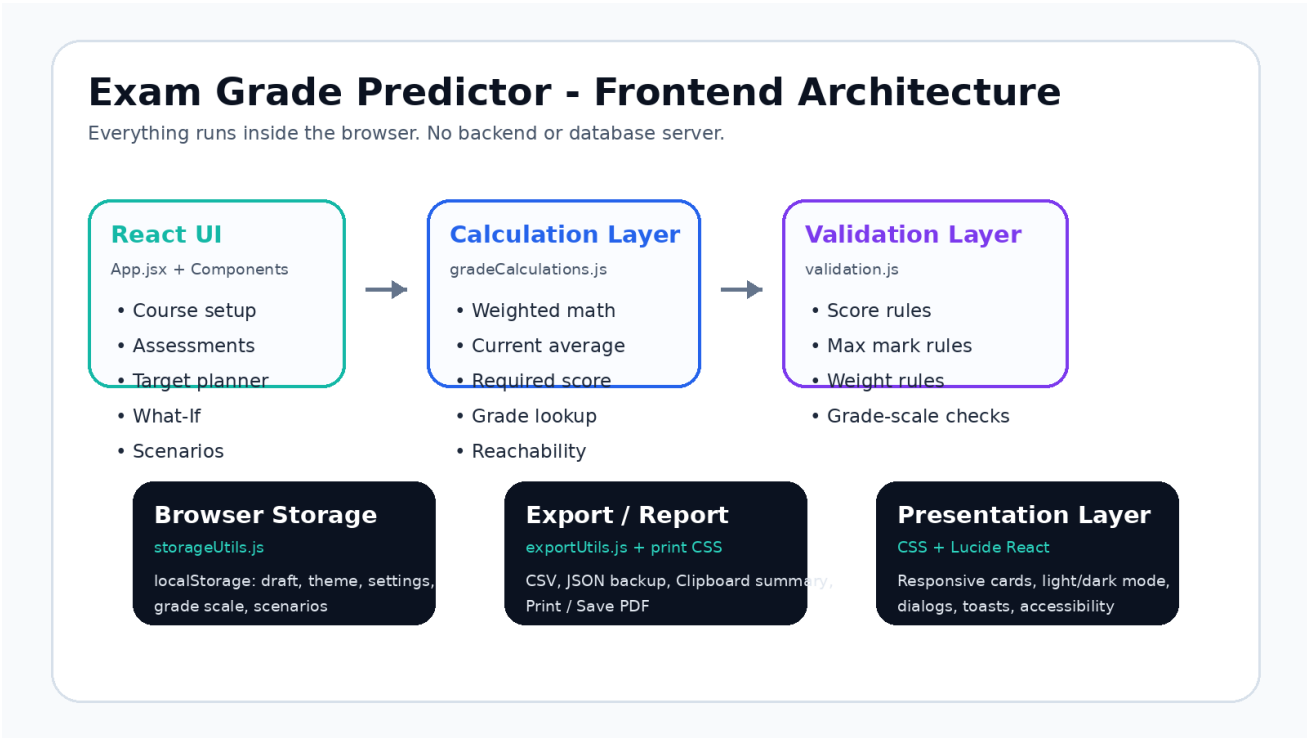
এই project-এর data sensitive server data না; এটা user-entered academic planning data. তাই server/API/database যোগ করলে complexity বাড়ত কিন্তু core learning value খুব বেশি বাড়ত না। localStorage দিয়েই browser-local persistence পাওয়া যায়।

Actual dependency note

The final package does not include Recharts or another charting library. Visual analytics are implemented with the existing React/CSS UI rather than an external chart dependency.

System Architecture & File Structure

How the codebase is organized and how data moves through it.



Data Model & Default Course Setup

What one assessment looks like in code and how the course model is built.

Assessment object

```
{
  id: 'midterm',
  name: 'Midterm',
  score: 24,
  maxScore: 30,
  weight: 30,
  status: 'completed'
}
```

এই object-টাই project-এর fundamental data unit. `id` stable identity, `name` display label, `score/maxScore` performance data, `weight` course contribution, আর `status` দিয়ে actual completed data ও pending planning data আলাদা করা হয়।

Editable default assessments

Assessment	Max Mark	Weight	Initial Status
Quiz	10	10%	Pending
Assignment	20	10%	Pending
Midterm	30	30%	Pending
Final Exam	50	50%	Pending

Not university-specific

These are editable example defaults. The project never claims that every university uses this weighting or grading system.

Example course used for demonstration

Assessment	Score	Max	Weight	Status
Quiz	8	10	10%	Completed
Assignment	18	20	10%	Completed
Midterm	24	30	30%	Completed
Final Exam	-	50	50%	Pending

06

Core Grade Mathematics

The formulas that make the application correct instead of being a simple marks average.

1. Assessment Percentage

ASSESSMENT PERCENTAGE

$$(\text{Score Earned} / \text{Maximum Score}) \times 100$$

Example: $8 / 10 \times 100 = 80\%$

এখানে raw mark-কে percentage-এ convert করা হয়। 8/10 এবং 40/50 - দুটোই 80%, কিন্তু course contribution depends on weight.

2. Weighted Contribution

WEIGHTED CONTRIBUTION

$$\text{Assessment Percentage} \times \text{Weight} / 100$$

Example: $80\% \times 10 / 100 = 8$ course points

Weighted contribution হলো full course-এর 100 points-এর মধ্যে ওই assessment কত point contribute করেছে। Quiz-এ 80% score হলেও যদি Quiz weight 10% হয়, contribution হবে 8 points - 80 course points না।

Worked example

Assessment	Percentage	Weight	Contribution
Quiz	80%	10%	8.00
Assignment	90%	10%	9.00
Midterm	80%	30%	24.00
Total completed	-	50%	41.00

Formula design decision

All important formulas live in `src/utils/gradeCalculations.js`, so components do not duplicate or slightly change the mathematics.

07

Current Average vs Course Points

This is the most important concept to understand for viva and correct grade planning.

CURRENT COURSE POINTS

Sum of completed weighted contributions

Example: $8 + 9 + 24 = 41$ course points

CURRENT AVERAGE

Course Points / Completed Weight x 100

Example: $41 / 50 \times 100 = 82\%$

এখানে একটা common ভুল হয়: Course Points = 41 দেখে কেউ যদি বলে current grade 41%, সেটা ভুল। কারণ student এখন পর্যন্ত course-এর শুধু 50% complete করেছে। Completed 50%-এর মধ্যে সে 41 points পেয়েছে, তাই completed-work average = 82%.

Metric	Meaning	Example
Course Points Earned	Full course-এর 100 points-এর মধ্যে ইতিমধ্যে earned points	41 / 100
Completed Weight	যে অংশের result actual completed	50%
Current Average	শুধু completed work-এর normalized average	82%
Remaining Weight	যে অংশ এখনো pending	50%

Viva-ready explanation

"41 is not my current average. It is my earned course points. Since only 50% of the course is completed, my current average is $41/50 \times 100 = 82\%$."

08

Target Planner & Required Score Logic

How the app answers “Final-এ কত পেলে target হবে?”

Target types

- Target Percentage - যেমন 80%, 75%, 50%.
- Target Letter Grade - যেমন A-, A, B+. Letter grade select করলে active grade scale-এর minimum threshold target percentage হিসেবে নেয়।

REQUIRED REMAINING AVERAGE

$$(\text{Target} - \text{Course Points}) / \text{Remaining Weight} \times 100$$

Use this when multiple assessments are still pending.

REQUIRED SELECTED ASSESSMENT %

$$(\text{Target} - \text{Course Points} - \text{Assumed Future Points}) / \text{Selected Assessment Weight} \times 100$$

Other pending assessments must have explicit What-If assumptions.

Example: Target A- (80%)

Demo course-এ Course Points = 41 এবং Final weight = 50%. অন্য কোনো pending assessment নেই।

REQUIRED FINAL

$$(80 - 41) / 50 \times 100 = 78\%$$

Final max mark 50 হলে required raw mark = 39 / 50

More targets with the same example

Target	Threshold	Required Final %	Required Raw (50)
Pass	50%	18%	9 / 50
B+	75%	68%	34 / 50
A-	80%	78%	39 / 50
A	85%	88%	44 / 50
A+	90%	98%	49 / 50

Why Math.ceil can be used

If the precise required raw mark is 35.2 and marks must be whole numbers, 35 is not enough. The display should say at least 36.

09

Multiple Pending Assessments & What-If Simulator

How the project avoids inventing assumptions.



Figure: Calculation pipeline and separation between actual and simulated data.

Why multiple pending assessments need assumptions

ধরো Project 20% এবং Final 40% - দুটোই pending। Final-এ কত লাগবে সেটা বের করতে Project-এ কত পাবে তার assumption দরকার। Project assumption না দিলে app Final-এর জন্য single number দেখায় না; instead missing assumption জানায়।

What-If Simulator

- Only Pending assessments appear in the simulator.
- Each pending assessment can use a slider, numeric percentage, or preset 50/60/70/80/90/100.
- Simulation values are stored separately in a `simulations` object keyed by assessment ID.
- Changing What-If values recalculates projected score, letter grade, pass/fail, and target gap.
- Reset What-If clears simulation values; it does not delete completed marks or saved scenarios.

```

simulations = {
  final: 80,
  project: 85
}

// completed assessment score is NOT modified
  
```

Example result

Demo course-এ Final What-If = 80% দিলে Final contribution = 40 points. Existing course points 41 + 40 = 81%. তাই projected final score 81% এবং default scale অনুযায়ী A-.

10

Best Case, Minimum Outcome & Grade Possibility

How the app turns mathematics into useful planning signals.

Best Possible Score

BEST POSSIBLE

Current Course Points + Pending Weight

Assumes 100% in every pending assessment; capped at 100 because no extra credit exists.

Demo course: 41 course points + 50 pending weight = 91%. তাই mathematical best case 91%, অর্থাৎ default scale-এ A+ reach করা possible, but near-perfect Final দরকার।

Minimum Mathematical Outcome

MINIMUM OUTCOME

Current Course Points

Assumes 0% in all remaining assessments; this is not a prediction.

Demo course-এর minimum mathematical outcome = 41%. এটা “predicted fail” বলা হয় না, কারণ student আসলে 0 পাবে এমন prediction app করে না।

Grade Possibility Status

Status	Rule	Meaning
Secured	Required average $\leq 0\%$	Completed points alone already meet threshold
Reachable	Required average > 0 and $< 100\%$	Target mathematically possible
Perfect Score Required	Required average = 100%	Only 100% across remaining work reaches it
Not Reachable	Required average $> 100\%$	Remaining weight cannot mathematically reach target

Target planner আরও user-friendly wording দেয়: comfortably achievable, achievable, requires strong performance - এগুলো deterministic planning labels, official academic standard নয়।

Custom Grade Scale & Pass/Fail Logic

Institution rules differ, so the project does not hardcode one universal grading policy.

Default example scale

Grade	Minimum %	Grade	Minimum %
A+	90	B-	65
A	85	C+	60
A-	80	C	55
B+	75	D	50
B	70	F	0

Grade scale validation

- Every grade needs a non-empty label and minimum percentage from 0 to 100.
- Grade labels must be unique.
- Minimum thresholds must be unique.
- A 0% floor is required so every score maps to a grade.
- Settings uses draft-and-save, so an invalid edit does not replace the active scale.

Pass/Fail

Pass mark configurable (default 50%). Final result known হলে score $\geq$ pass mark হলে Pass, otherwise Fail. Course incomplete হলে app final Pass/Fail claim না করে current threshold/performance wording ব্যবহার করে।

Subtle but important

Current Average 82% does not automatically mean “pass secured.” If only 41 full-course points are earned, future zeros could still keep the final result below a 50% pass mark.

React State, Hooks & Application Logic

How the interface stays synchronized with the mathematics.

Important state in App.jsx

State	Purpose
theme	Light/dark mode
gradeScale	Active custom grade thresholds
settings	Whole raw marks + decimal precision
scenarios	Saved planning snapshots
courseName / courseCode	Course identity
passMark	Pass threshold
assessments	Main course model
targetType / targetGrade / targetPercentage	Target configuration
simulations	Pending What-If percentages
targetAssessmentId	Which pending assessment should be solved for
toasts / confirm / modal state	UX feedback and dialogs

Why useMemo?

`coursePoints`, `completedWeight`, `pendingWeight`, `currentAverage`, target, projection, bestPossible, gradePossibilities - এগুলো input state থেকে derive হয়। `useMemo` ব্যবহার করে এগুলো cleanly calculate করা হয়েছে, যাতে formula logic UI event handler-এর মধ্যে ছড়িয়ে না যায়।

Why useEffect?

Theme, grade scale, settings, scenarios, data version এবং current draft localStorage-এ persist করার side effect handle করতে useEffect ব্যবহৃত হয়েছে। ফলে refresh করলেও browser-local state recover করা যায়।

Design principle

Store inputs; derive results. Calculated values should be recomputed from the source data instead of manually keeping many duplicated result states.

Saved Scenarios & localStorage

How the project persists planning data without a database.

Storage keys

```
examGradePredictor.scenarios  
examGradePredictor.gradeScale  
examGradePredictor.settings  
examGradePredictor.theme  
examGradePredictor.draft  
examGradePredictor.dataVersion
```

`safeParse()` JSON parse error catch করে fallback return করে। তাই malformed localStorage app crash করায় না। `readStorage`, `writeStorage`, `removeStorage` browser restrictions/error handle করার জন্য try/catch ব্যবহার করে।

What a saved scenario keeps

- Course name and course code
- Assessment snapshot
- Grade-scale snapshot
- Pass mark
- Target type/value
- What-If assumptions
- Projected result when valid
- Save date and scenario name

Why grade-scale snapshot matters

আজ A- threshold 80%, পরে user যদি global scale change করে A- = 82% করে, পুরোনো scenario silently reinterpret করা ঠিক না। তাই scenario save করার সময় grading scale snapshot রাখা হয় এবং recover/import-এর সময় saved data থেকে outcome re-derive করা হয়।

Scenario actions

Save, Load, Duplicate, Rename, Delete এবং up to 3 scenario comparison implement করা আছে। Delete destructive action custom confirmation dialog ব্যবহার করে।

Export, Backup, Clipboard & Print

How browser APIs turn the planner into a usable portfolio application.

CSV Export

Assessment, Score, Maximum Score, Percentage, Weight, Contribution এবং Status column export হয়। Fields double-quoted, internal quotes escaped, আর download-এ UTF-8 BOM যোগ করা হয় যাতে spreadsheet software-এ Unicode compatibility ভালো থাকে।

JSON Backup / Import

Saved scenarios, grade scale এবং settings JSON backup হিসেবে download করা যায়। Import-এর আগে basic schema এবং grade-scale validity check হয়; invalid JSON বা malformed data app crash করে না।

Copy Summary

Clipboard API ব্যবহার করে course summary copy করা যায় - current average, completed weight, target, projected result, required selected score ইত্যাদি।

Print / Save as PDF

`@media print` rules editing controls, navigation, theme buttons ইত্যাদি hide করে A4-friendly report তৈরি করে। User browser Print -> Save as PDF ব্যবহার করতে পারে।

Why this improves portfolio value

The project is not only a calculator. It demonstrates browser storage, file export, print styling, Clipboard API, custom dialogs, and resilient data handling.

Validation, Precision & Edge Cases

The logic that prevents polished-looking but mathematically invalid results.

Assessment validation rules

Input	Rule
Name	Cannot be blank
Maximum score	Must be > 0
Weight	0 to 100
Completed score	0 to maximum score
Pending score	Actual score may stay blank
Status	Completed or Pending

Weight model rules

Total Weight	State	Behavior
90%	Incomplete	Warn about 10% unassigned; do not normalize
100%	Valid	Final model can be interpreted normally
110%	Over 100%	Block final/reachability interpretation until corrected
Malformed	Invalid	Prompt user to fix weight inputs

Floating-point tolerance

JavaScript floating-point math-এ 79.99999999 এর মতো tiny error হতে পারে। `EPSILON = 1e-9` এবং helper functions threshold boundary-তে tolerance ব্যবহার করে, যাতে mathematically exact 80 unexpectedly lower grade না হয়।

Major edge cases covered

- No assessments / no completed weight
- Score greater than max
- Negative score or weight
- Max score = 0
- Only Final pending
- Multiple pending rows with missing assumptions
- Target already secured
- Exact 100% requirement
- Required > 100%
- No remaining weight
- Malformed grade scale
- Corrupted localStorage
- CSV fields with commas/quotes/Unicode

UI/UX, Responsive Design & Accessibility

How the application moves beyond a spreadsheet-style calculator.

Visual direction

UI academic dashboard style: clear result hierarchy, panels/cards, subtle borders/shadows, target and grade emphasis, light/dark theme, custom toast/dialog components. Primary objective হলো user যেন কয়েক সেকেন্ডেই current performance, target এবং required score বুঝতে পারে।

Responsive strategy

CSS breakpoints at approximately 1120px, 900px, 700px, and 460px cover the requested desktop/tablet/mobile targets. Desktop-এ assessment table, small screen-এ same data mobile assessment cards হিসেবে render হয় - squeezed horizontal table না।

Accessibility measures

- Visible labels and semantic section headings
- aria-label on icon buttons and numeric controls
- aria-invalid for invalid assessment fields
- aria-pressed for presets/selection states
- Accessible slider labels and value text
- Status communicated with text, not color only
- Dialogs support focus containment, Escape, and focus restoration
- Custom confirmation dialogs instead of browser alert()

Testing & Final Audit

What was verified and what the audit tells you about software quality.

Calculation test command

```
npm run test:audit
```

The regression suite uses Node `assert` and directly tests the centralized utility functions. This is important because UI can look correct while formulas are wrong; testing the math layer catches those errors.

Verified areas

- Weighted contribution and invalid input rejection
- Current Average vs Course Points
- Full-course score
- 90/100/110 weight states
- Required remaining and selected-assessment formulas
- Raw mark upward rounding
- Missing pending assumptions
- Secured / exact 100 / impossible target logic
- What-If non-mutation
- Best/minimum bounds
- Grade boundary tolerance
- Custom grade-scale validation
- Pass threshold tolerance
- CSV escaping
- localStorage fallback behavior

Static and code-quality audit

Final audit record reports clean JavaScript/JSX parsing, CSS parsing, and source scans for `console.log`, `Math.random`, browser `alert()`, and fragile Final-name matching. Core calculation logic does not depend on assessment name; the default Final uses stable ID `final` only for default selection behavior.

Build audit note

In the audit sandbox, npm registry DNS returned EAI_AGAIN, so dependency install and Vite build could not be honestly marked as executed there. On a normal machine, run `npm install -> npm run test:audit -> npm run build`.

How to Run, Build & Deploy

The practical commands you need on a Windows development machine.

Requirements

Node.js 20.19+ or 22.12+ (or a newer compatible release), npm, VS Code or another editor, and a modern browser.

Run locally

```
cd F:\Projects\exam-grade-predictor
npm install
npm run dev
```

Vite terminal-এ local URL দেখাবে (commonly `http://localhost:5173`). Browser-এ URL open করলে app run করবে।

Production verification

```
npm run test:audit
npm run build
npm run preview
```

GitHub + Vercel deployment summary

1. Push the project repository to GitHub (for example: mrinamoto/exam-grade-predictor).
2. In Vercel choose Add New -> Project and import that GitHub repository.
3. Framework preset: Vite.
4. Build command: npm run build.
5. Output directory: dist.
6. Install command: npm install.
7. No environment variables are required for the current frontend-only project.
8. Deploy and test the generated public URL on desktop and mobile.

Do not commit generated folders

Keep `node\_modules/` and usually `dist/` out of Git. They are regenerated from package.json during install/build.

How the Project Idea Becomes a Working Application

A practical mental model for how this project could be planned from zero.

Step 1 - Identify the real student problem

Idea শুরু হয় marks calculator দিয়ে না; problem statement দিয়ে: students weighted grading বুঝতে ভুল করে এবং future target manually calculate করতে অসুবিধা হয়। তাই project-এর value হলো planning + transparency.

Step 2 - Define mathematical truths

- Every assessment has max score and course weight.
- Completed work contributes course points.
- Current average must normalize by completed weight.
- Pending work cannot be treated as actual marks.
- Targets must respect remaining weight and reachability.

Step 3 - Convert math into a data model

Each assessmentকে object বানানো হয়, তারপর array of assessments পুরো course represent করে। This makes add/remove/edit dynamic and keeps formulas generic.

Step 4 - Separate layers

UI components input/display handle করে; utility modules math/validation/storage/export handle করে। এই separation debugging, testing আর viva explanation সহজ করে।

Step 5 - Add real product behavior

Basic calculation কাজ করার পর target planner, What-If, grade ladder, scenarios, dark mode, responsive cards, import/export, print report, accessibility যোগ করা হয়েছে - এগুলো project-কে portfolio-quality application বানায়।

Step 6 - Audit edge cases

Finally, exact 100%, impossible targets, malformed grade scale, score > max, overweight course, floating-point boundary, corrupted localStorage-এর মতো cases test করা হয়েছে।

What You Learn From This Project

The technical and problem-solving skills this project demonstrates.

Skill	What you practiced in this project
React fundamentals	Components, props, state, controlled inputs, conditional rendering
Hooks	useState, useMemo, useEffect, useRef
Data modeling	Dynamic assessment objects and arrays
Business logic	Weighted formulas, target solving, grade lookup, reachability
Validation	Field errors, model validity, grade-scale rules
Browser storage	Safe localStorage persistence and versioned keys
Browser APIs	Blob download, Clipboard API, print
UX design	Dashboard hierarchy, feedback, confirmations, empty states
Responsive CSS	Desktop table -> mobile cards
Accessibility	ARIA, focus behavior, status text
Testing mindset	Regression tests for formulas and boundaries
Deployment	Vite build, GitHub repository, Vercel hosting

Biggest conceptual learning

Software project মানে শুধু UI বানানো না। Correct data model + verified formulas + validation + state management + persistence + edge cases + user communication - সব একসাথে reliable product তৈরি করে। এই project-এ সবচেয়ে valuable learning হলো “business logic কে UI থেকে আলাদা রেখে testable করা।”

Limitations & Future Improvements

What the current version intentionally does not do, and where it can grow.

Current limitations

- Results are only as accurate as user-entered marks, weights, and status.
- Institution grading policies differ; default scale is example only.
- No official university/LMS integration.
- What-If is mathematics, not a prediction of how well the student will actually perform.
- Saved data exists only in the current browser; clearing localStorage may remove it.
- No cross-device synchronization.
- No extra-credit model; score cannot exceed maximum marks.
- No user accounts or cloud database.

Good future extensions

- Multiple courses in one semester dashboard
- GPA/CGPA integration
- Optional account + cloud sync
- Institution-specific reusable templates
- LMS/CSV import
- Richer analytics/history charts
- More automated unit/integration tests
- PWA/offline install support
- Shareable read-only scenario links

Scope discipline

Not adding a backend in Version 1 is not a weakness. It keeps the project focused on the academic-planning problem and makes deployment simple.

How to Present This Project in Portfolio or Viva

A concise explanation you can give without memorizing the whole codebase.

30-second project explanation

Presentation script

"Exam Grade Predictor is a frontend-only React application that models a course using weighted assessments. It calculates current average and earned course points separately, lets students set a target grade, solves the required score across remaining assessments, and includes a What-If simulator. I centralized all grade formulas in utility functions, added validation, localStorage-based scenarios, custom grade scales, export/print, responsive design, accessibility, and regression tests."

What to show in a live demo

9. Load Example data.
10. Point out Current Average = 82% vs Course Points = 41.
11. Set target A- and show required Final = 78% / 39 out of 50.
12. Move Final What-If to 80% and show projected result = 81% (A-).
13. Change target to A+ and discuss strong requirement / reachability.
14. Open grade scale settings and explain customization.
15. Save two scenarios and compare them.
16. Switch mobile responsive view or theme.
17. Export CSV or Print Report.

Questions an evaluator may ask

- Why not simply average all percentages?
- Why is 41 course points not 41% current average?
- Why localStorage instead of database?
- How do you prevent invalid scores?
- How do multiple pending assessments work?
- What happens if required score is 120%?
- How do saved scenarios preserve historical grade meaning?

Viva Questions & Answers

Short, technically correct answers based on the actual final implementation.

1. What is a weighted grade?

A weighted grade means each assessment contributes according to its course weight, not equally.

2. Why can't we simply average Quiz, Midterm and Final percentages?

Because their weights may be different. Equal averaging would ignore the course structure.

3. What is weighted contribution?

Assessment percentage multiplied by its weight divided by 100. It represents course percentage points earned from that assessment.

4. What is the difference between Current Average and Course Points?

Course Points are points earned out of the full course 100. Current Average normalizes those points by completed course weight.

5. How is Current Average calculated?

Current Course Points / Completed Weight x 100.

6. How is the required Final score calculated?

(Target - existing course points - assumed future points) / selected assessment weight x 100.

7. Why are other pending assumptions required?

Because their future contribution changes how much the selected assessment must contribute.

8. What happens if the required score is over 100%?

The target is marked not achievable through the remaining assumptions instead of showing an impossible score as normal.

9. What does exact 100% mean?

It is achievable but requires a perfect score, so the app labels it Perfect Score Required.

10. What does Target Already Secured mean?

Completed course points already meet the target even before any additional contribution is needed.

11. How does What-If analysis work?

Pending percentage assumptions are stored separately and passed to projection functions; actual completed scores are not modified.

12. Why is the grading scale customizable?

Different institutions use different thresholds, so one fixed scale would be misleading.

13. Why must the grade scale include a 0% threshold?

So every possible score maps to a grade without a gap.

14. How is pass/fail decided?

When a valid final/projected score is known, it is compared with the configured pass mark using tolerance.

15. Why can a high Current Average still not mean Pass is secured?

Because the completed weight may be small; future zero scores could still pull the full-course result below the pass mark.

16. Why use localStorage?

The project is frontend-only and the planning data can be persisted in the browser without a server or database.

17. What are the limitations of localStorage?

Data is browser/device-local and can be lost if storage is cleared; there is no cross-device sync.

18. How does the app survive corrupted localStorage?

JSON parsing is wrapped in safe fallback logic so malformed data returns defaults instead of crashing.

19. Why use React?

The app has many interactive inputs and derived values. React components and hooks keep state and UI synchronized.

20. Why use useMemo?

To derive grade calculations from input state cleanly and avoid duplicating calculation logic across event handlers.

21. What does useEffect do here?

It persists theme, settings, grade scale, scenarios, and draft data to localStorage when relevant state changes.

22. Why are formulas centralized?

One source of truth prevents inconsistent calculations and makes the math directly testable.

23. How is score > max prevented?

Validation flags it and calculation helpers return null instead of computing an invalid percentage.

24. What is floating-point tolerance?

A tiny epsilon is used near boundaries so values like 79.999999999 do not unexpectedly miss a mathematically exact threshold.

25. How is required raw mark rounded?

If whole-number marks are enabled, Math.ceil is used so the displayed minimum is actually sufficient.

26. How do saved scenarios remain historically meaningful?

Each scenario stores a snapshot of the grade scale and assumptions used when it was saved.

27. What can be exported?

Assessment breakdown CSV, JSON backup, clipboard summary, and an A4-friendly print report.

28. Is this an AI prediction system?

No. It is deterministic mathematics based on entered and simulated marks.

29. Does the project use a database?

No. It is frontend-only and uses browser localStorage.

30. What is the biggest improvement over a basic calculator?

It combines correct weighted mathematics, pending-work planning, What-If simulation, reachability, persistence, and product-quality UX.

Final Understanding Checklist

If you can explain these points, you understand the project well enough for portfolio and viva.

- I can explain why weighted contribution is different from raw percentage.
- I can calculate 8/10 at 10% weight = 8 course points.
- I can explain 41 course points from 50% completed weight = 82% current average.
- I can derive a required Final score from target, earned points and Final weight.
- I understand why multiple pending assessments need explicit assumptions.
- I know What-If values do not overwrite completed marks.
- I can explain React component/state structure and centralized utilities.
- I know what localStorage stores and its limitations.
- I know how grade-scale validation and floating-point tolerance protect correctness.
- I can run, test, build and deploy the project.
- I can explain at least five limitations and future improvements.
- I can demo the project in under 3 minutes.

Portfolio conclusion

This project demonstrates more than grade calculation: it combines mathematics, state management, validation, persistence, export/reporting, responsive UX, accessibility, and testing in one coherent student-focused product.

Created by Ann Noor Muhammad Mrinmoy

Exam Grade Predictor - Project Documentation & Learning Report